

MaixCAM MaixPy 图像基础操作

2024-07-08

▼ 更新历史

日期	版本	作者	更新内容
2024-07-08	1.1.0	neucrack	优化 cv 和 image 转换文档
2024-04-03	1.0.0	neucrack	初版文档

一、简介

视觉应用中图像占据非常重要的位置，不管是图片还是视频，因为视频本质上就是一帧帧的图像，所以图像处理是视觉应用的基础。

二、API 文档

本文介绍常用方法，更多 API 参考 [maix.image](#) 模块的文档。

三、图像格式

MaixPy 提供基础图像模块 `image`，里面最重要的就是 `image.Image` 类，用于图像的创建以及各种图像基础操

作，以及图像加载和保存等。

图像格式有很多，一般我们用 `image.Format.FMT_RGB888` 或者 `image.Format.FMT_RGBA8888` 或者 `image.Format.FMT_GRAYSCALE` 或者 `image.Format.FMT_BGR888` 等。

大家知道 `RGB` 三色可以合成任意颜色，所以一般情况下我们使用 `image.Format.FMT_RGB888` 就足够，`RGB888` 在内存中是 `RGB packed` 排列，即在内存中的排列：

像素1_红色，像素1_绿色，像素1_蓝色，像素2_红色，像素2_绿色，像素2_蓝色，... 依次排列。

四、创建图像

创建图像很简单，只需要指定图像的宽度和高度以及图像格式即可：

```
1  from maix import image
2
3  img = image.Image(320, 240,
4  image.Format.FMT_RGB888)
5  print(img)
   print(img.width(), img.height(),
   img.format())
```

`320` 是图像的宽度，`240` 是图像的高度，`image.Format.FMT_RGB888` 是图像的格式，格式参数可以省略，默认是 `image.Format.FMT_RGB888`。

这里通过 `img.width()`、`img.height()`、`img.format()` 可以获取图像的宽度、高度和格式。

五、显示到屏幕

MaixPy 提供了 `maix.display.Display` 类，可以方便的显示图像：

```
1 from maix import image, display
2
3 disp = display.Display()
4
5 img = image.Image(320, 240,
6 image.Format.FMT_RGB888)
disp.show(img)
```

注意这里因为没有图像数据，所以显示的是黑色的图像，修改画面看后文。

六、从文件系统读取图像

MaixPy 提供了 `maix.image.load` 方法，可以从文件系统读取图像：

```
1 from maix import image
2
3 img = image.load("/root/image.jpg")
4 if img is None:
5     raise Exception(f"load image
6 failed")
print(img)
```

注意这里 `/root/image.jpg` 是提前传输到了板子上的，方法可以看前面的教程。

可以支持 `jpg` 和 `png` 格式的图像。

七、保存图像到文件系统

MaixPy 的 `maix.image.Image` 提供了 `save` 方法，可以保存图像到文件系统：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5
6 # do something with img
img.save("/root/image.jpg")
```

八、画框

`image.Image` 提供了 `draw_rect` 方法，可以在图像上画框：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
img.draw_rect(10, 10, 100, 100,
image.Color.from_rgb(255, 0, 0))
```

这里的参数依次是：`x`，`y`，`w`，`h`，`color`，`x` 和 `y` 是框的左上角坐标，`w` 和 `h` 是框的宽度和高度，`color` 是框的颜色，可以使用 `image.Color.from_rgb` 方法创建颜色。

可以用 `thickness` 指定框的线宽，默认是 `1`，

也可以画实心框，传参 `thickness=-1` 即可：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
img.draw_rect(10, 10, 100, 100,
(255, 0, 0), thickness=-1)
```

九、写字符串

`image.Image` 提供了 `draw_string` 方法，可以在图像上写字：

```
1 | from maix import image
2 |
3 | img = image.Image(320, 240,
4 | image.Format.FMT_RGB888)
   img.draw_string(10, 10, "Hello
   MaixPy", image.Color.from_rgb(255,
   0, 0))
```

这里的参数依次是：`x`，`y`，`text`，`color`，`x` 和 `y` 是文字的左上角坐标，`text` 是要写的文字，`color` 是文字的颜色，可以使用 `image.Color.from_rgb` 方法创建颜色。

还可以放大字体，传参 `scale` 即可：

```
1 | img.draw_string(10, 10, "Hello
   MaixPy", image.Color.from_rgb(255,
   0, 0), scale=2)
```

获取字体的宽度和高度：

```
1 | w, h = image.string_size("Hello
2 | MaixPy", scale=2)
   print(w, h)
```

注意这里 `scale` 是放大倍数，默认是 `1`，和 `draw_string` 应该保持一致。

十、中文支持和自定义字体

`image` 模块支持加载 `ttf/otf` 字体，默认字体只支持英文，如果要显示中文或者自定义字体可以先下载字体文件到设备上，然后加载字体。

系统也内置了几个字体，在 `/maixapp/share/font` 目录下面，代码示例：

```
1  from maix import image, display,
2  app, time
3
4  image.load_font("sourcehansans",
5  "/maixapp/share/font/SourceHanSansCN-
6  Regular.otf", size = 32)
7  print("fonts:", image.fonts())
8  image.set_default_font("sourcehansans")
9
10 disp = display.Display()
11
12 img = image.Image(disp.width(),
13 disp.height())
14 img.draw_string(2, 2, "你好! Hello,
    world!", image.Color.from_rgba(255,
    0, 0, 0.8))
    disp.show(img)
    while not app.need_exit():
        time.sleep(1)
```

加载字体文件，然后设置默认的字体，也可以不设置默认的字体，在写字的函数参数设置：

```
1  | img.draw_string(2, 2, "你好! Hello,
    world!", image.Color.from_rgba(255,
    0, 0, 0.8), font="sourcehansans")
```

注意 `string_size` 方法也会使用设置的默认字体计算大小，也可以通过 `font` 参数单独设置要计算大小的字体。

十一、画线

`image.Image` 提供了 `draw_line` 方法，可以在图像上画线：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
img.draw_line(10, 10, 100, 100,
image.Color.from_rgb(255, 0, 0))
```

这里的参数依次是：`x1`，`y1`，`x2`，`y2`，`color`，`x1` 和 `y1` 是线的起点坐标，`x2` 和 `y2` 是线的终点坐标，`color` 是线的颜色，可以使用 `image.Color.from_rgb` 方法创建颜色。

十二、画圆

`image.Image` 提供了 `draw_circle` 方法，可以在图像上画圆：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
img.draw_circle(100, 100, 50,
image.Color.from_rgb(255, 0, 0))
```

这里的参数依次是：`x`，`y`，`r`，`color`，`x` 和 `y` 是圆心坐标，`r` 是半径，`color` 是圆的颜色，可以使用 `image.Color.from_rgb` 方法创建颜色。

十三、缩放图像

`image.Image` 提供了 `resize` 方法，可以缩放图像：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img_new = img.resize(160, 120)
  print(img, img_new)
```

注意这里 `resize` 方法返回一个新的图像对象，原图像不变。

十四、剪裁图像

`image.Image` 提供了 `crop` 方法，可以剪裁图像：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img_new = img.crop(10, 10, 100, 100)
  print(img, img_new)
```

注意这里 `crop` 方法返回一个新的图像对象，原图像不变。

十五、旋转图像

`image.Image` 提供了 `rotate` 方法，可以旋转图像：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img_new = img.rotate(90)
  print(img, img_new)
```


注意这里 `rotate` 方法返回一个新的图像对象，原图像不变。

十六、拷贝图像

`image.Image` 提供了 `copy` 方法，可以拷贝一份独立的图像：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img_new = img.copy()
  print(img, img_new)
```

十七、仿射变换

`image.Image` 提供了 `affine` 方法，可以进行仿射变换，即提供当前图中三个及以上的点坐标，以及目标图中对应的点坐标，可以自动进行图像的旋转、缩放、平移等操作变换到目标图像：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img_new = img.affine([(10, 10),
  (100, 10), (10, 100)], [(10, 10),
  (100, 20), (20, 100)])
  print(img, img_new)
```

更多参数和用法请参考 API 文档。

十八、画关键点

`image.Image` 提供了 `draw_keypoints` 方法，可以在图像上画关键点：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5
6 keypoints = [10, 10, 100, 10, 10,
7             100]
8 img.draw_keypoints(keypoints,
9 image.Color.from_rgb(255, 0, 0),
10 size=10, thickness=1, fill=False)
```

在坐标 `(10, 10)`、`(100, 10)`、`(10, 100)` 画三个红色的关键点，关键点的大小是 `10`，线宽是 `1`，不填充。

十九、画十字

`image.Image` 提供了 `draw_cross` 方法，可以在图像上画十字：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img.draw_cross(100, 100,
6 image.Color.from_rgb(255, 0, 0),
7 size=5, thickness=1)
```

在坐标 `(100, 100)` 画一个红色的十字，十字的延长大小是 `5`，所以线段长度为 `2 * size + thickness`，线宽是 `1`。

二十、画箭头

`image.Image` 提供了 `draw_arrow` 方法，可以在图像上画箭头：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
img.draw_arrow(10, 10, 100, 100,
image.Color.from_rgb(255, 0, 0),
thickness=1)
```

在坐标 `(10, 10)` 画一个红色的箭头，箭头的终点是 `(100, 100)`，线宽是 `1`。

二十一、画图

`image.Image` 提供了 `draw_image` 方法，可以在图像上画图：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img2 = image.Image(100, 100,
6 image.Format.FMT_RGB888)
img2.draw_rect(10, 10, 90, 90,
image.Color.from_rgb(255, 0, 0))
img.draw_image(10, 10, img2)
```

二十二、转换格式

`image.Image` 提供了 `to_format` 方法，可以转换图像格式：

```
from maix import image
```

```
3   img = image.Image(320, 240,  
4   image.Format.FMT_RGB888)  
5   img_new =  
6   img.to_format(image.Format.FMT_BGR888)  
7   print(img, img_new)  
   img_jpg =  
   img.to_format(image.Format.FMT_JPEG)  
   print(img, img_new)
```

注意这里 `to_format` 方法返回一个新的图像对象，原图像不变

二十三、 maix.image.Image 对象和 Numpy/OpenCV 格式互相 转换

见 [MaixPy 使用 OpenCV 文档](#)

二十四、和 bytes 数据互相 转换

`image.Image` 提供了 `to_bytes` 方法，可以转换图像为 `bytes` 数据：

```
from maix import image  
  
img = image.Image(320, 240,  
image.Format.FMT_RGB888)  
data = img.to_bytes()  
print(type(data), len(data),  
img.data_size())  
  
img_jpeg = image.from_bytes(320,  
240, image.Format.FMT_RGB888, data)
```

```
print(img_jpeg)
img =
img_jpeg.to_format(image.Format.FMT_RGB)
print(img)
```

这里 `to_bytes` 获得一个新的 `bytes` 对象，是独立的内存，不会影响原图。

`image.Image` 构造函数中传入 `data` 参数可以直接从 `bytes` 数据构造图像对象，注意新的图像也是独立的内存，不会影响到 `data`。

因为涉及到内存拷贝，所以这个方法比较耗时，不建议频繁使用。

如果你想用不拷贝的方式优化程序（不建议轻易使用，写不好代码会导致程序容易崩溃，），请看 API 文档。

二十五、更多基础 API 使用方法

更多 API 使用方法请参考 `maix.image` 模块的文档。